Week 8: Genetic Algorithm

CLO - 2: Apply classical AI techniques, including uninformed and informed search algorithms, minimax, and neural networks, to solve structured problems

Lab Objectives

By completing this lab, students will be able to:

1. Understand the fundamental GA concepts: **chromosome representation, population, fitness, selection, crossover, mutation**.
2. Implement GA for **binary-encoded and combinatorial (permutation-based) optimization problems**.
3. Analyze the effect of **GA parameters** (population size, mutation rate, selection type) on convergence.
4. Apply GA to **realistic problem scenarios**.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Genetic Algorithm for Maximizing the Number of 1s (Basic GA Concepts)

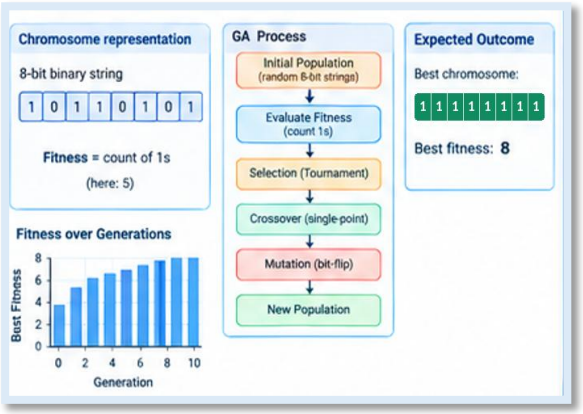
A Genetic Algorithm can represent a solution using a binary chromosome, where each gene contains either 0 or 1. The fitness of a chromosome can be calculated by counting the number of 1s.

Implement a Genetic Algorithm to find an 8-bit chromosome containing the maximum possible number of 1s.

Requirements

1. Generate an initial population of 10 binary chromosomes of length 8.
2. Calculate the fitness of each chromosome as the number of 1s.
3. Select parents using **Tournament Selection**.
4. Apply **Single-Point Crossover** with a crossover probability of 0.8.
5. Apply bit-flip mutation with a mutation probability of 0.02 per gene.

6. Generate a new population from the offspring.
7. Repeat the process for a maximum of 50 generations.
8. Display:
    - Best chromosome
    - Best fitness
    - Best fitness in each generation
9. Stop early if a chromosome with fitness 8 is found.



Expected Outcome:

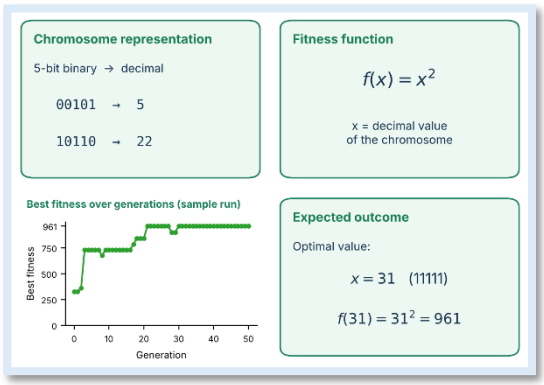
The algorithm should converge toward: [1, 1, 1, 1, 1, 1, 1, 1] with a fitness of: 8

Task 2: Function Optimization Using Genetic Algorithm

A Genetic Algorithm can also be used to optimize a mathematical function.

Consider the following function:

$$f(x) = x^2$$



where:

$$0 \leq x \leq 31$$

Represent each value of x using a **5-bit binary chromosome**.

For example:

00101 → 5

10110 → 22

Implement a Genetic Algorithm to find the value of x that maximizes:

$$f(x) = x^2$$

Requirements

1. Represent each candidate value of x as a 5-bit chromosome. Use a population of 10 chromosomes.

2. Decode each chromosome into its decimal value.
3. Calculate its fitness using:

$$Fitness(x) = x^2$$

4. Use **Roulette Wheel Selection**, with a fallback to **uniform random selection** if the total population fitness is zero.

   Note: Since (x^2) is non-negative, it can be used directly as fitness, but if every individual has zero fitness, roulette-wheel probabilities are undefined.

5. Apply **Single-Point Crossover**.
6. Apply **Bit-Flip Mutation**.
7. Generate a new population for each generation.
8. Record the best x and its fitness in every generation.
9. Run the algorithm for 50 generations.
10. Display the final best solution.

Expected Objective

The optimal value is:

x = 31

because:

$$31^2 = 961$$

The purpose of the task is to demonstrate how GA searches the solution space rather than directly calculating the mathematical maximum.

Task 3: Travelling Salesperson Problem Using Genetic Algorithm

Objective

Use Genetic Algorithm to search for a near-optimal travelling route while maintaining valid permutation chromosomes.

Problem Statement:

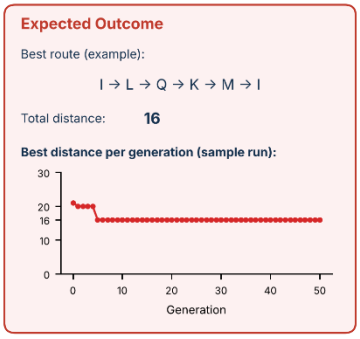
The Travelling Salesperson Problem (TSP) requires a salesperson to visit every city exactly once and return to the starting city while minimizing the total travel distance.

Consider the following cities:

Islamabad (I), Lahore (L), Karachi (K), Multan (M), Quetta (Q)

and the following distance matrix:

From/To	I	L	K	M	Q
I	0	4	7	3	6
L	4	0	5	6	2
K	7	5	0	4	3
M	3	6	4	0	5
Q	6	2	3	5	0



Represent each solution as a **permutation chromosome**.

For example:

[I, K, Q, L, M]

represents the route:

I → K → Q → L → M → I

Implement a Genetic Algorithm to find a route with **minimum** total distance.

Requirements

1. Generate an initial population of 10 valid routes.
2. Ensure that:
    - Every city appears exactly once.
    - The route returns to the starting city.
3. Calculate the total route distance.
4. Use an appropriate fitness formulation for a minimization problem.
5. Use **Tournament Selection**.
6. Apply Order Crossover (OX) or another permutation-preserving crossover operator. The offspring must contain every city exactly once. Ordinary single-point crossover must not be used unless additional logic guarantees valid permutations.
7. Apply mutation by swapping two cities.

8. Verify that every offspring and mutated chromosome contains each city exactly once. The route must include the return edge from the final city to the starting city when calculating total distance.
9. Generate new populations for 50 generations.
10. Display:
    - Best route
    - Total distance
    - Best distance per generation
11. Plot the final route using a simple graph visualization.

Task 4: 0/1 Knapsack Problem using GA

Problem: Optimize selection of items within weight limit.

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	8

- Max weight = 8

Instructions:

1. **Chromosome:** Binary string of length 4 (1 = item selected). Use a population of 10 chromosomes.
2. **Fitness:** Calculate the total value and total weight of the selected items. If total weight is less than or equal to 8, fitness equals total value; otherwise, fitness equals 0.
3. **Selection:** Use Roulette Wheel Selection. If all chromosomes have zero fitness, select parents uniformly at random until a feasible chromosome becomes available.
4. **Crossover:** Single-point crossover.
5. **Mutation:** Flip one random bit.
6. **Generations:** 10 generations.
7. Preserve the best feasible solution found so far, if one exists, so that it is not lost during population replacement.

Output:

- Best chromosome, total weight, total value per generation.

Hints:

- Calculate the total weight first, then assign **fitness 0 if the weight limit is exceeded**.

Expected Outcome:

The best feasible solution is [0, 1, 0, 1] (items 2 and 4), with total weight 8 and total value 12.

Task 5: N-Queens Problem Using Genetic Algorithm

Objective

Use Genetic Algorithm to evolve a population of possible queen arrangements toward a conflict-free solution.

Problem Statement:

The N-Queens problem requires placing N queens on an N × N chessboard so that no two queens attack each other.

For this task, solve the **8-Queens problem** using a Genetic Algorithm.

Solve the problem in two parts. Part A uses a permutation chromosome. Part B uses a plain digit-string chromosome, as shown in the lecture.

Represent a chromosome as a sequence in which each position represents a column and the value represents the row containing the queen.

For example: [1, 5, 8, 6, 3, 7, 2, 4]

represents one possible arrangement of eight queens.

Part A: Permutation Chromosome

Chromosome Validity

Represent a solution as a permutation of the integers 1 through 8, where the position represents the column and the value represents the row. This representation guarantees that exactly one queen occupies each row and each column. Crossover and mutation must preserve this permutation property. Suitable operators include Order Crossover (OX) and swap mutation.

Requirements

1. Generate an initial population of 50 candidate solutions.
2. Represent each candidate as an 8-element chromosome.
3. Calculate fitness as the number of pairs of queens that do not attack each other. Since the chromosome guarantees one queen per row and column, only diagonal conflicts need to

be checked. The maximum fitness is 28, because 8 queens form 8 × 7 / 2 = 28 pairs, and in a solution all 28 pairs are non-attacking.

4. Select parents using Tournament Selection, choosing the individual with the highest fitness.
5. Apply a permutation-preserving crossover operator.
6. Apply swap mutation by exchanging the row values at two randomly selected column positions.
7. Generate a new population.
8. Continue until:
    - A valid 8-Queens solution is found, or
    - The maximum of 500 generations is reached.
9. Display:
    - Best chromosome
    - Fitness
    - Number of generations
10. Visualize the final chessboard arrangement.

Expected Outcome

Best chromosome (example):

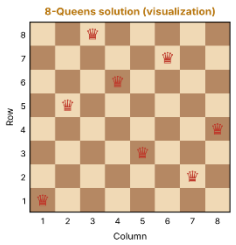
```
[1, 5, 8, 6, 3, 7, 2, 4]
```

Fitness:

28 (no conflicts)

Generations:

varies from run to run



Part B: Plain Digit-String Chromosome (as in the lecture)

Represent a solution as a string of 8 digits, where the position represents the column and the digit (1 to 8) represents the row. Digits may repeat, as in the lecture example 24748552, so two queens can be in the same row. This chromosome is not a permutation, so ordinary single-point crossover can be used.

Requirements

1. Generate an initial population of 100 chromosomes. Each gene is a random digit from 1 to 8.
2. Calculate fitness as the number of pairs of queens that do not attack each other. Check both row conflicts (two equal digits) and diagonal conflicts. The maximum fitness is 28.
3. Select parents using Tournament Selection.
4. Apply Single-Point Crossover with a crossover probability of 0.8.
5. Apply mutation: replace each gene with a random digit from 1 to 8, with a mutation probability of 1/8 per gene.

6. Copy the best chromosome of each generation unchanged into the next generation (elitism).
7. Continue until a chromosome with fitness 28 is found or the maximum of 1000 generations is reached.
8. Display the best chromosome, its fitness and the number of generations.
9. Visualize the final chessboard arrangement.
10. Compare Part A and Part B: the number of generations each needs, and why Part A only has to check diagonal conflicts.

Expected Outcome:

The lecture chromosome 24748552 has fitness 24. A solution has fitness 28, for example [7, 4, 2, 5, 8, 1, 3, 6].

Task 6: Smart Delivery Route Optimization: Real-World Application(TSP)

Objective

Demonstrate how Genetic Algorithms can be applied to a real-world route optimization problem where the objective is to minimize travel distance.

Problem Statement:

A food delivery company wants to optimize the route followed by a delivery rider.

The rider starts from a restaurant and must deliver orders to several customers before returning to the restaurant.

Consider the following locations:

Restaurant, Customer A, Customer B, Customer C, Customer D, Customer E

The distance between locations is provided in the following matrix:

From/To	**R**	**A**	**B**	**C**	**D**	**E**
R	0	4	7	6	5	8
A	4	0	3	5	6	4
B	7	3	0	4	5	6
C	6	5	4	0	3	5
D	5	6	5	3	0	4

E	8	4	6	5	4	0

Use a Genetic Algorithm to find a route that **minimizes** the total travel distance.

Requirements

1. Represent each route as a permutation chromosome containing Customer A, Customer B, Customer C, Customer D and Customer E exactly once. The restaurant must not be included as a gene.
2. Keep the Restaurant fixed as the starting and ending location. Calculate each route as Restaurant → all customers in the chromosome order → Restaurant.
3. Generate an initial population of 20 valid routes.
4. Calculate total delivery distance.
5. Apply Tournament Selection.
6. Use permutation-preserving crossover and swap mutation. Verify that every customer occurs exactly once in each offspring and mutated chromosome.
7. Run the algorithm for 100 generations.
8. Display:
   - Best delivery route
   - Total distance
   - Fitness improvement across generations
9. Compare the **initial** best route with the **final** best route.

Expected Outcome:

The minimum total distance is 25, for example Restaurant → A → B → C → E → D → Restaurant (or the same route in reverse).

****